Log Analytics System for a Web Server
Review 2

HDFS · Apache Spark · Spark SQL · Spark MLlib · Apache Superset

Faculty Name: Dr. Senthil Kumar K
Team Member Info:
Kushal Mohan – RA2411028010082
Kriti Singh – RA2411028010081
Gargi Singh – RA2411028010102
Big Data Essentials · Department of Networking and Communications

Contents

1. Dataset
2. Proposed Model
3. Overall System Architecture Diagram
4. Preprocessing (Big Data Analysis and its steps)
5. Results of Preprocessing (graphs and tables)
6. Classification (proposed model: Factorization Machine)
7. Results of Classification using the proposed model
8. Comparison with the existing classifier models
9. Visualization (Apache Superset + LogLens app)
10. Plan of Action / Member Tasks
11. References
12. Geotagged photo taken during Review 2
13. Google Colab Implementation link

Dataset

NASA-HTTP (main)

Real web-server access logs of NASA Kennedy Space Center

- July + August 1995 (58 days)
- 3,461,613 requests, 356 MB
- 137,978 client hosts
- Unlabelled: used for traffic analytics, anomalies and bots

HttpParams (labelled)

31,067 HTTP parameter values

- normal, SQL injection, XSS, command injection, path traversal
- Used to train and test attack detection

Fwaf (labelled)

1,342,657 URLs

- 1,294,531 normal
- 48,126 malicious
- Used to train and test attack detection

One raw NASA log line (Common Log Format) and its 7 fields

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
```

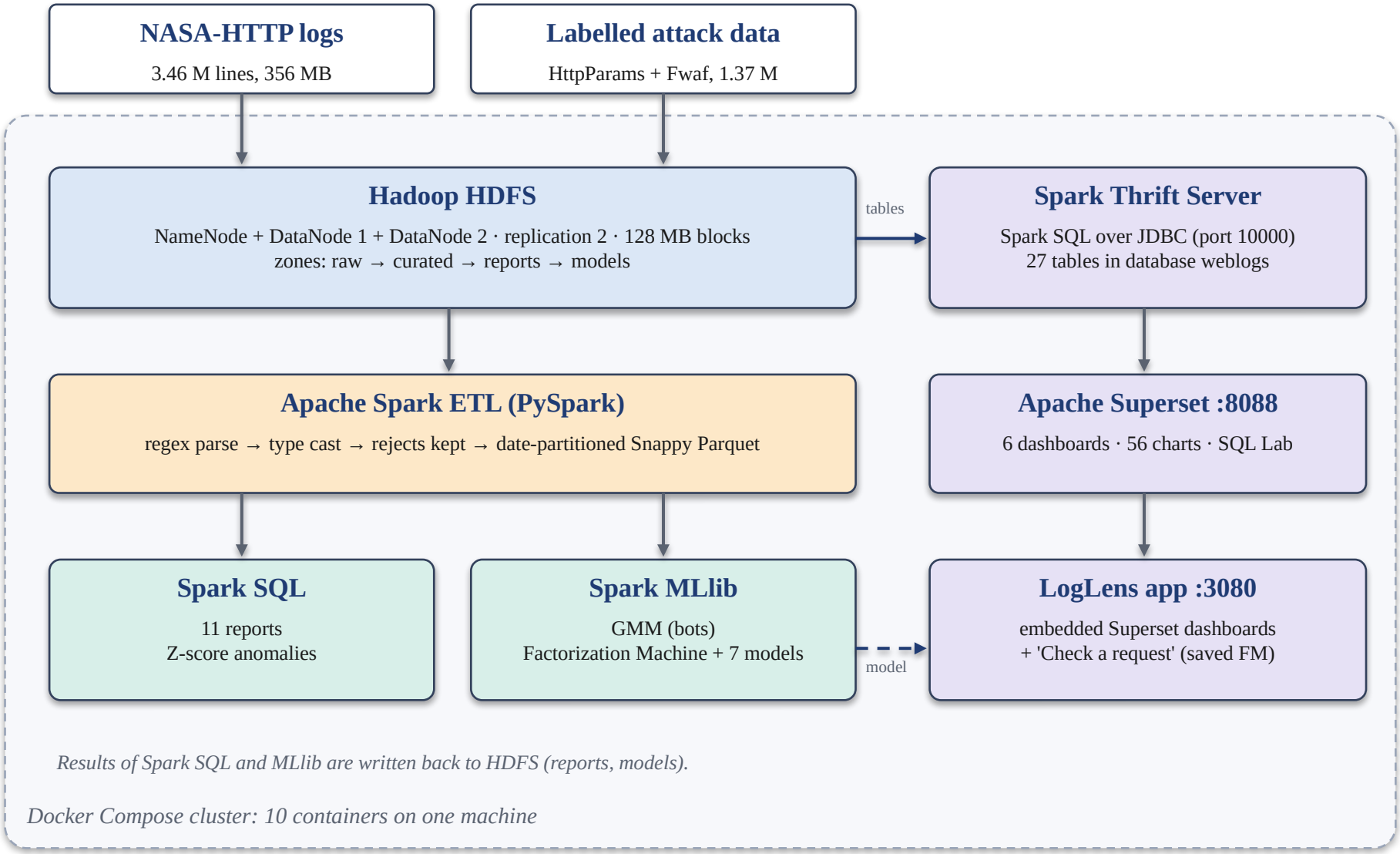
host	timestamp	method	URL (endpoint)	protocol	status	bytes
199.72.81.55	01/Jul/1995:00:00:01 -0400	GET	/history/apollo/	HTTP/1.0	200	6245

Learning type: supervised binary classification, 2 classes (0 = normal, 1 = attack) on the labelled data; unsupervised clustering (bots) and statistics (Z-score) on the unlabelled NASA logs.

Proposed Model

1	When was traffic unusual?	**Contextual Z-score (Spark SQL)** Each hour is compared with the same hour of day on all 58 days; |z| > 3 is flagged.
2	Which clients are bots?	**Gaussian Mixture Model (Spark MLlib, unsupervised)** 9 behaviour features per host, 3 components; the one with HEAD requests, slow pacing and few images is 'automated'.
3	Is a request an attack?	**Factorization Machine (Spark MLlib, supervised) ★ proposed classifier** Character 1-3-grams (12,292 features); learns a weight for every pair of features. Compared with 7 MLlib models.
4	How do users see it?	**Apache Superset + LogLens app** Superset dashboards on the Spark Thrift Server, embedded in the LogLens application with an attack checker.

Overall System Architecture Diagram



Preprocessing: Big Data Analysis Steps

1	**Ingest**	hdfs dfs -put: logs into the HDFS raw zone, 128 MB blocks, 2 copies
2	**Read**	Spark reads the raw zone as one text column, split into partitions
3	**Parse**	5 regular expressions cut each line into 7 fields
4	**Cast**	timestamp (New York time), status → int, bytes → long ('-' = 0)
5	**Validate**	lines without host/time/status → rejects folder, parse rate printed
6	**Derive**	date, hour, day of week, file type
7	**Store**	Snappy Parquet, partitioned by date → HDFS curated zone
8	**Analyse**	11 Spark SQL reports, Z-score, GMM → HDFS reports zone

loganalytics — Spark ETL run on the full dataset

```
$ spark-submit jobs/etl.py data curated
[etl] read 3,461,613 raw lines from data
[etl] {
  "raw_lines": 3461613,
  "clean_records": 3461612,
  "malformed": 1,
  "parse_rate_pct": 100.0,
  "date_min": "1995-07-01",
  "date_max": "1995-08-31",
  "output": "curated"
}
$ ls curated | head -n 4
_SUCCESS
date=1995-07-01
date=1995-07-02
date=1995-07-03
$ ls curated | grep -c date=
58
$ cat curated_rejects/part-*.txt
alyssa.p
```

Real run of the ETL on the cluster: 3,461,613 lines in, 1 malformed line kept for audit.

Preprocessing for Classification

Step	What happens	Rows
1-2 Load + label	HttpParams + Fwaf, attack types → 1 class	1,373,724
3 Clean	URL-decode, lower-case, trim, drop empty	1,373,657
3b Bias fix	keep only the part after '?'	—
4 De-duplicate	66,987 duplicates removed	1,306,518
5 Conflicts	501 requests with both labels removed	1,306,017
7 Split 80/20	train balanced 4 normal : 1 attack	153,847
	test with the real mix (never trained on)	261,076

Final dataset: 1,306,017 requests

1,267,605 normal (97.06%) · 38,412 attack (2.94%)

All steps run in Spark (jobs/ml_preprocess.py); data read from and written to HDFS.

Why the bias fix?

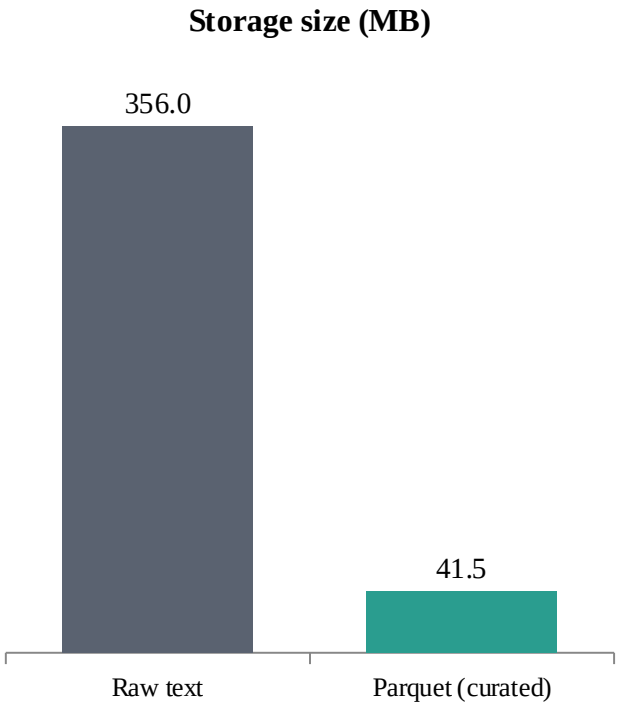
Before: 42.2% of attacks but only 2.5% of normal requests contained '?'.
A model could learn the shortcut "has a query string = attack".
Fix: '/top.php?stuff=1 or 1=1' → 'stuff=1 or 1=1'
After: 2.7% vs 0.8%; the model must read what is sent.

Features (12,292)

character 1-, 2-, 3-grams, each hashed into 4,096 buckets, + length, special chars, digits, attack keywords; scaled 0-1

Results of Preprocessing: Raw Logs → Table

3,461,612	1	58	8.6×
clean rows (of 3,461,613)	malformed line, kept as a reject	date partitions (Parquet)	smaller: 356 MB → 41.5 MB



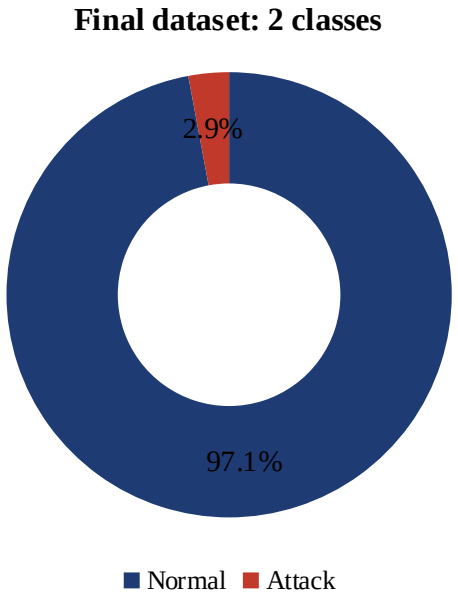
loganalytics — cleaned data in the curated zone

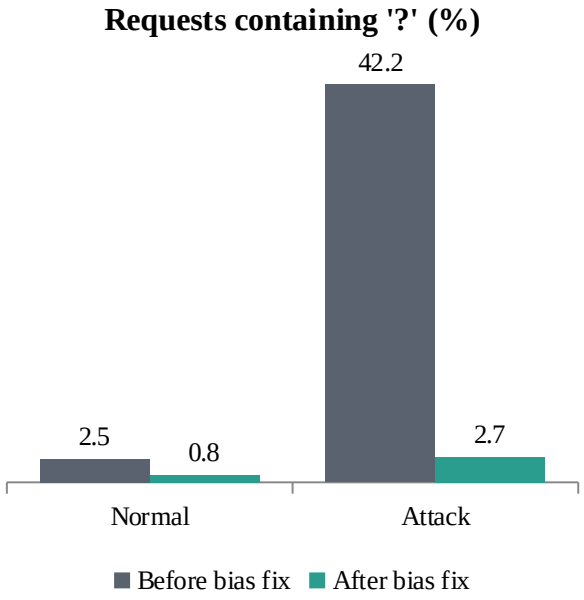
```
$ cat peek.py
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate(); spark.sparkContext.setLogLevel("ERROR")
df = spark.read.parquet("curated")
df.printSchema()
df.show(6)
print(df.count())
$ spark-submit peek.py
root
 |-- host: string (nullable = true)
 |-- method: string (nullable = true)
 |-- endpoint: string (nullable = true)
 |-- protocol: string (nullable = true)
 |-- status: integer (nullable = true)
 |-- bytes: long (nullable = true)
 |-- ts: timestamp (nullable = true)
 |-- hour: integer (nullable = true)
 |-- dow: string (nullable = true)
 |-- ext: string (nullable = true)
 |-- date: date (nullable = true)

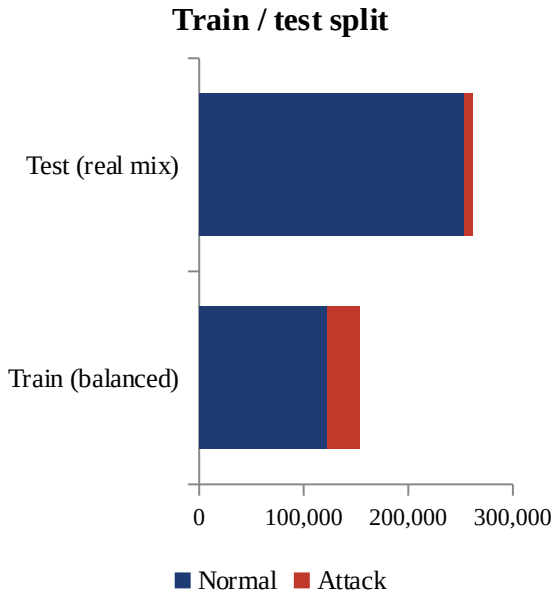
+------------------+------+------------------+--------+------+-----+-------------------+----+--------+----+----------+
|              host|method|          endpoint|protocol|status|bytes|                 ts|hour|     dow| ext|      date|
+------------------+------+------------------+--------+------+-----+-------------------+----+--------+----+----------+
|      pc1.flex.co.jp|   GET|/shuttle/missions...|HTTP/1.0|   200| 8678|1995-07-13 00:00:02|   0|Thursday|html|1995-07-13|
|       ns2.cti.co.jp|   GET|                 /|HTTP/1.0|   200| 7062|1995-07-13 00:00:04|   0|Thursday|    |1995-07-13|
|www-d4.proxy.aol.com|   GET| /htbin/cdt_clock.pl|HTTP/1.0|   200|  752|1995-07-13 00:00:05|   0|Thursday|  pl|1995-07-13|
|      pc1.flex.co.jp|   GET|/images/launchmed...|HTTP/1.0|   200|11853|1995-07-13 00:00:05|   0|Thursday| gif|1995-07-13|
|port-3-7.access.o...|   GET|/shuttle/missions...|HTTP/1.0|   200| 5949|1995-07-13 00:00:06|   0|Thursday|html|1995-07-13|
|     141.248.128.110|   GET|/software/winvn/f...|HTTP/1.0|   200| 1991|1995-07-13 00:00:07|   0|Thursday|html|1995-07-13|
+------------------+------+------------------+--------+------+-----+-------------------+----+--------+----+----------+
only showing top 6 rows
3461612
```

After: the same requests as typed columns (host, method, endpoint, protocol, status, bytes, ts, hour, dow, ext, date).

Results of Preprocessing: Labelled Data







	Normal	Attack	Total	Purpose
Training set	123,100	30,747	153,847	all attacks + 4 normal per attack
Test set	253,411	7,665	261,076	real 97 : 3 mix, never seen in training

Screenshot of the full run: docs/ml_results/ml_preprocessing.png (every step prints its row counts).

Classification: Proposed Model (Factorization Machine)

$$\hat{y}(x) = \sigma\left(w_0 + \Sigma_i w_i x_i + \Sigma_i \Sigma_{j>i} \langle v_i, v_j \rangle x_i x_j \right)$$

bias + one weight per feature (like logistic regression) + a weight for every PAIR of features, factorised through 8-dimensional vectors v_i

Why a Factorization Machine?

- Attacks are combinations: a quote AND 'or' AND a comment marker
- Learns pair weights even for pairs never seen together
- Linear time O(k·n) on sparse input; built into Spark MLlib

Settings (chosen on a validation split)

- factor size k = 8, step 0.2 (best of $k \in \{4, 8, 16\}$, step $\in \{0.05, 0.2\}$)
- 100 iterations, L2 = 10^{-4}; validation F1 0.988
- test set never used for tuning

Pipeline inside Spark ML (saved to HDFS as one model for the app)



Task: supervised binary classification, 2 classes (normal / attack). Main metric: F1, because 97% of requests are normal.

Results of Classification using the Proposed Model

0.9669	98.5%	95.0%	99.80%
F1 (attack class)	attacks caught (recall)	precision	accuracy

Confusion matrix (test set, 261,076 requests)

	Predicted normal	Predicted attack
Actual normal	**253,012** TN · 99.84%	**399** FP · 0.16%
Actual attack	**118** FN · 1.54%	**7,547** TP · 98.46%

Result from the run on the Docker cluster (jobs/ml_train.py). The saved model, reloaded from HDFS, gives the same F1 = 0.9669.

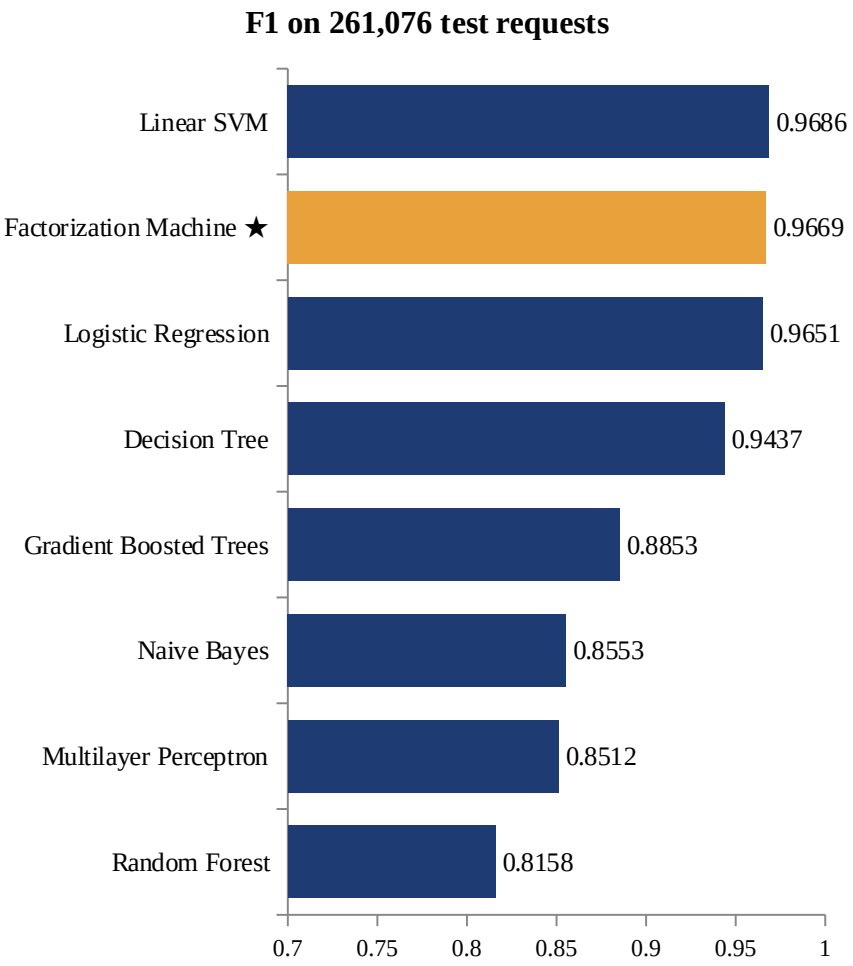
What it means

- 7,547 of 7,665 attacks caught
- 399 false alarms among 253,411 normal requests (0.16%)
- 118 attacks missed

On the real NASA logs

- 31,026 URLs scored, 120 flagged (0.05% of requests)
- Checked by hand: 0 real attacks; mostly searches with 'select' and pages named 'description'

Comparison with the Existing Classifier Models

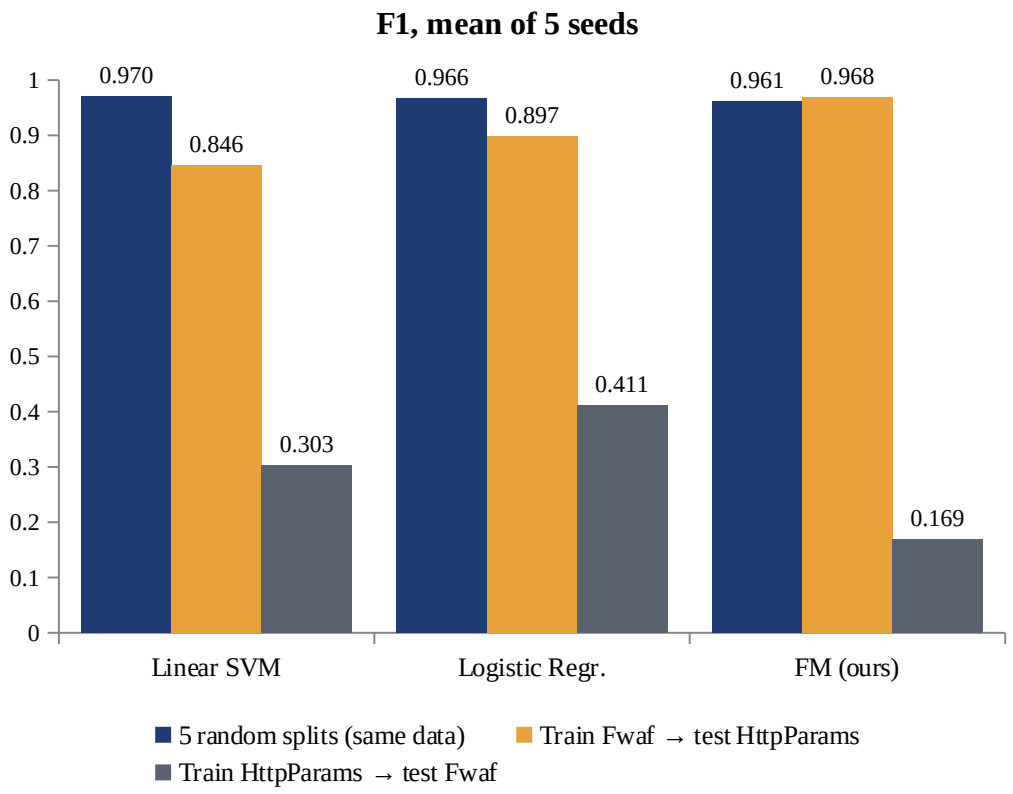


Model	Prec.	Recall	F1	Time (s)
Linear SVM	0.949	0.989	0.9686	5
Factorization Machine ★	**0.950**	**0.985**	**0.9669**	**67**
Logistic Regression	0.946	0.986	0.9651	4
Decision Tree	0.929	0.959	0.9437	27
Gradient Boosted Trees	0.819	0.963	0.8853	132
Naive Bayes	0.915	0.803	0.8553	0
Multilayer Perceptron	0.779	0.939	0.8512	44
Random Forest	0.957	0.711	0.8158	24

- Linear SVM, FM and Logistic Regression lead; trees, Naive Bayes and the neural network trail.
- Accuracy is above 99% for all 8 and does not separate them; F1 does (0.816 – 0.969).

★ = our model.

Comparison: Stability and New Data



Inside one dataset

- SVM 0.9697 ± 0.0007
- FM 0.9611 ± 0.0062
- McNemar's test: SVM better on 4 of 5 splits ($p < 0.05$)

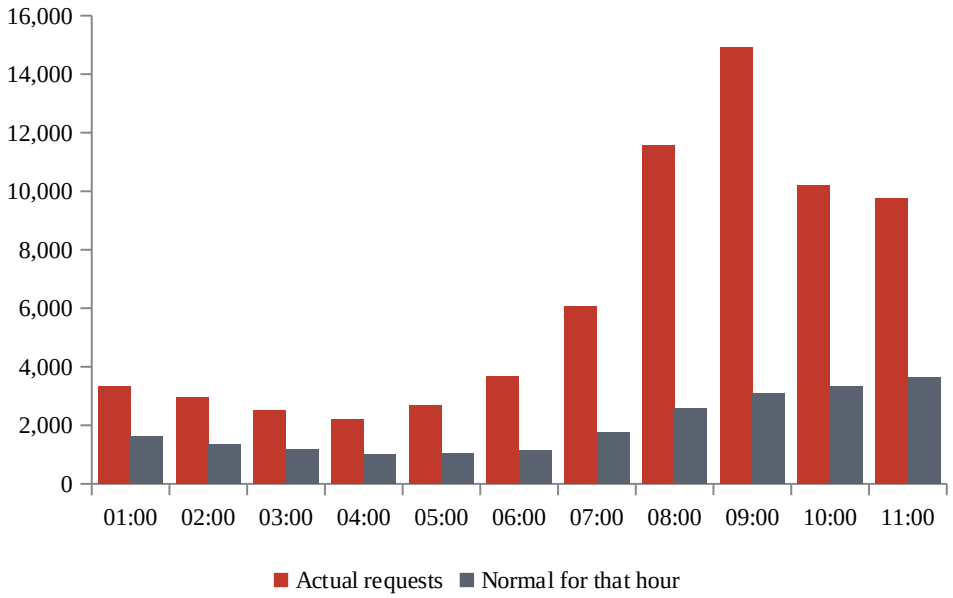
On new data (Fwaf → HttpParams)

- FM 0.968 vs LR 0.897 vs SVM 0.846
- 6-14× fewer false alarms than LR and SVM

Conclusion: the SVM is the most accurate inside one dataset; the FM generalises best to data it was not trained on, so the FM is the deployed model. With only the small HttpParams set to learn from, every model fails.

Big Data Analysis Results: Anomalies and Bots

13 July 1995: 11 of the 12 unusual hours (|z| > 3)



- Peak: 09:00-10:00, 14,926 requests vs a normal 3,080: z = 5.90, the STS-70 launch at 9:41 a.m.
- Only anomalous day: 13 July (z = 3.62).

Bot detection: GMM, 3 components

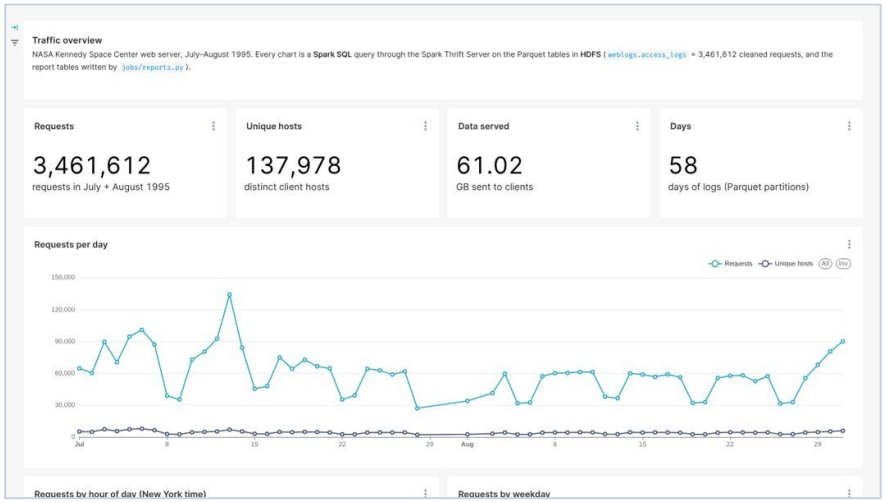
	Hosts	HEAD %	Gap (s)	Images %
C0 human	32,503	0.04	5	63.2
C1 human	3,772	0.10	2	70.2
C2 automated	**1,829**	**2.50**	**18**	**33.5**

6.36%
of requests are automated

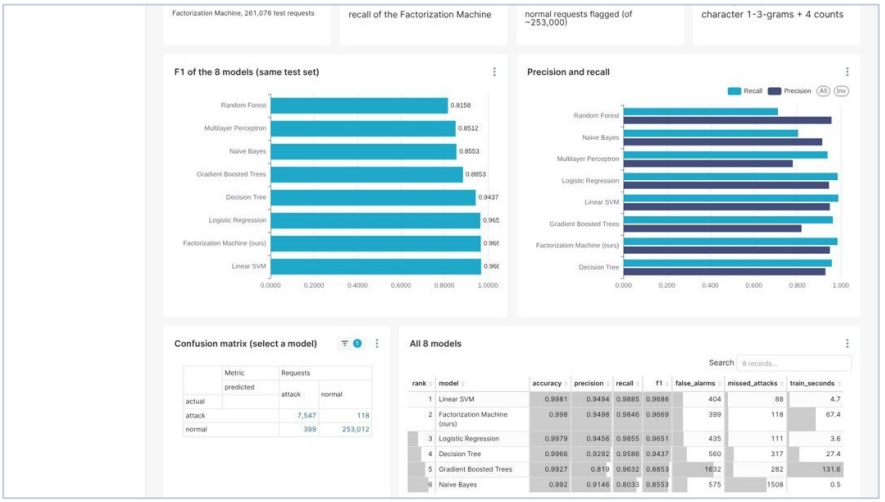
11.6×
lift on robots.txt crawlers

- 9 features per host: HEAD share, pacing, images, sessions, URLs, errors…
- 15 of 27 known crawlers fall in C2 (no labels used to train).
- GMM beats K-Means: the bot group is small and elongated.

Visualization: Apache Superset Dashboards



1 · Traffic overview dashboard



5 · Attack detection (ML) dashboard

HDFS

Parquet tables

→

Spark Thrift Server

Spark SQL over JDBC :10000

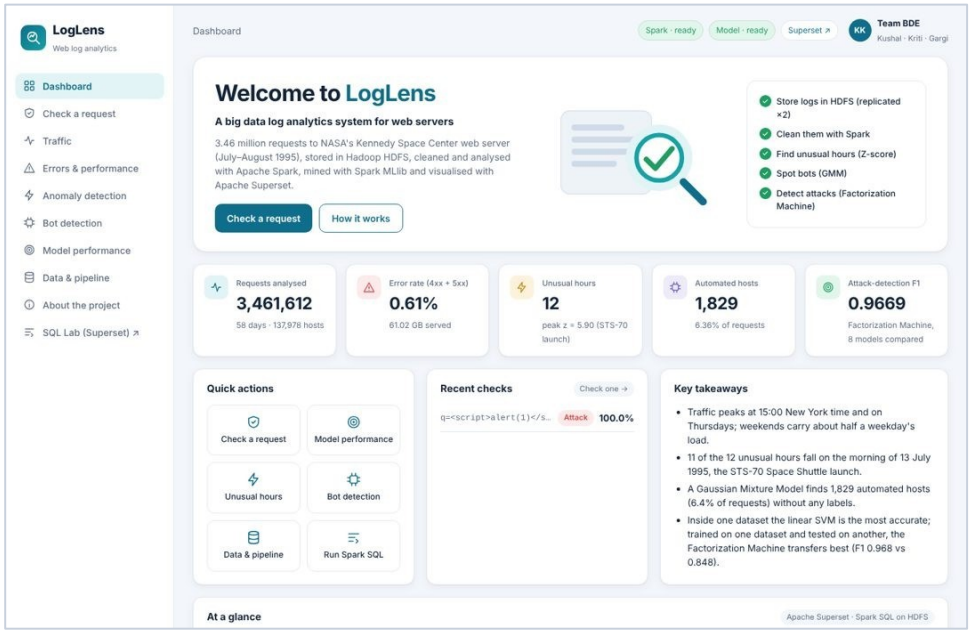
→

Apache Superset

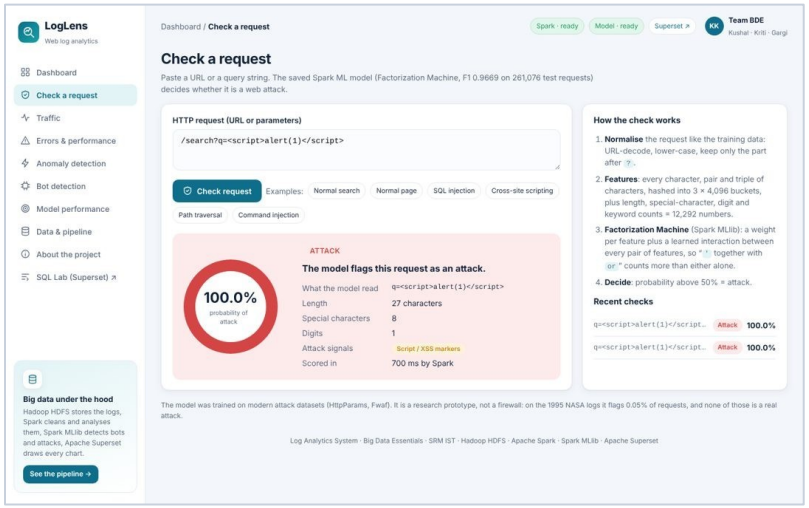
6 dashboards · 56 charts :8088

- Dashboards: Home · Traffic · Errors & performance · Anomalies (Z-score) · Bot detection (GMM) · Attack detection (ML)
- Nothing is copied out of HDFS; filters, cross-filters and SQL Lab work live on the cluster.
- All dashboards are created from code (superset/build_dashboards.py), so they can be rebuilt any time.

Visualization: LogLens Application



Home: live numbers from HDFS + embedded Superset dashboard



Check a request: the saved FM model

- Pages: Dashboard · Check a request · Traffic · Errors · Anomalies · Bots · Model performance · Data & pipeline
- Every chart is an embedded Superset dashboard (guest token for one dashboard only)
- Runs in Docker at http://localhost:3080

Example

/search?q=<script>alert(1)</script>
→ ATTACK, 100.0% probability, signal: script / XSS marker, scored by Spark in ~0.7 s
/htbin/wais.pl?apollo+13+mission → NORMAL, 0.04%

Plan of Action

Phase 1	**Cluster + ingest**	Docker cluster, HDFS with 2 DataNodes, raw logs loaded and replicated	Done
Phase 2	**Preprocessing**	Spark ETL to date-partitioned Parquet; labelled data cleaned, bias fixed, split	Done
Phase 3	**Analysis**	11 Spark SQL reports, Z-score anomalies, GMM bot detection	Done
Phase 4	**Classification + visualization**	Factorization Machine vs 7 models; Thrift Server, Superset dashboards, LogLens app	Done
Phase 5	**Final review**	CSIC 2010 data, hand-label flagged NASA URLs, average several FM runs, 5× / 10× scale test	Next

Member Tasks: Team Work Split

Kriti Singh

RA2411028010081

Storage layer: Hadoop HDFS + Docker

- Docker Compose cluster (10 containers)
- HDFS: NameNode + 2 DataNodes, replication 2
- Ingest into the raw zone, fsck checks
- Windows / Mac runners

Kushal Mohan

RA2411028010082

Processing layer: Apache Spark

- Spark ETL: regex parsing, type casting
- Rejects kept, date-partitioned Parquet
- ML preprocessing: cleaning, bias fix
- De-duplication, train / test split

Gargi Singh

RA2411028010102

Analysis + BI layer: Spark SQL, Superset

- 11 Spark SQL reports
- Z-score anomaly detection
- Spark Thrift Server tables
- 6 Apache Superset dashboards

Together (all three members)

- Spark MLlib: GMM bot detection
- Factorization Machine vs 7 models, tuning, 5-split stability, cross-dataset test
- LogLens app (embedded Superset + attack checker)
- 35 automated tests · IEEE paper · this presentation

References

[1] K. Shvachko et al., "The Hadoop distributed file system," IEEE MSST, 2010, pp. 1–10.

[2] M. Zaharia et al., "Resilient distributed datasets," USENIX NSDI, 2012, pp. 15–28.

[3] M. Armbrust et al., "Spark SQL: Relational data processing in Spark," ACM SIGMOD, 2015, pp. 1383–1394.

[4] X. Meng et al., "MLlib: Machine learning in Apache Spark," JMLR, vol. 17, no. 34, pp. 1–7, 2016.

[5] S. Rendle, "Factorization machines," IEEE ICDM, 2010, pp. 995–1000.

[6] D. A. Reynolds, "Gaussian mixture models," Encyclopedia of Biometrics, Springer, 2009, pp. 659–663.

[7] M. F. Arlitt and C. L. Williamson, "Internet web servers: Workload characterization and performance implications," IEEE/ACM Trans. Networking, vol. 5, no. 5, 1997.

[8] V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, 2009.

[9] R. Geirhos et al., "Shortcut learning in deep neural networks," Nature Machine Intelligence, vol. 2, 2020.

[10] Q. McNemar, "Note on the sampling error of the difference between correlated proportions," Psychometrika, vol. 12, 1947.

[11] Internet Traffic Archive, "NASA-HTTP: Two months of HTTP logs from the KSC-NASA WWW server."

[12] Morzeux, "HttpParamsDataset," GitHub; F. Ahmad, "Fwaf: Machine learning driven web application firewall," GitHub, 2017.

[13] Apache Software Foundation, "Apache Superset documentation" and "Spark SQL Thrift JDBC/ODBC server," Spark 3.5.3 docs.

Geotagged Photo taken during Review 2

Insert the geotagged photo here

(photo with location and date stamp, taken during the Review 2 presentation)

Google Colab Implementation Link



Google Colab notebook (scan or click):

https://colab.research.google.com/github/kushal040511/Web-server-log-analytics-system/blob/combined-project/notebooks/LogAnalytics_Colab.ipynb

Source code (GitHub):

https://github.com/kushal040511/Web-server-log-analytics-system

- Notebook: installs Java, Hadoop and Spark, starts HDFS, runs the ETL, the reports, the Z-score and the ML stage (Runtime → Run all)
- Full system on the Docker cluster: make data build up all ml bi